

Refatoração em Python

Cheiros, Refatorações e a Disciplina do Ciclo Red–Green–Refactor

Prof. Dr. André Yoshiaki Kashiwabara

DCOMP — Universidade Federal de Sergipe

`andreyoshiaki@dcomp.ufs.br`

Problema: *enxergar* o que está ruim — cheiros de código



Solução: *consertar* com refatorações nomeadas



Disciplina: *sustentar* com Red-Green-Refactor, testes e PEP 8

O que são *code smells*

Problema:

Código *funciona* mas é difícil de manter, estender ou entender. Bugs aparecem em cascata, mudanças simples viram epopeias.

Solução:

Aprender a **farejar** sinais comuns de degradação — e ter uma refatoração específica pronta para cada um.

Definição (Kent Beck & Martin Fowler)

Um *code smell* é uma estrutura no código que **sugere** — mas não prova — a necessidade de refatoração. Não é bug: o código funciona, mas algo está *cheirando mal*.

Intuição — “If it stinks, change it”

O nariz é treinável. Quanto mais código você lê, mais rápido percebe que algo está errado — antes mesmo de saber explicar o quê. Smells são *atalhos heurísticos*.

Bug

Código se comporta errado.
Testes falham (ou deveriam).

Urgência: alta.

Smell

Código se comporta certo,
mas é frágil para mudar.

Urgência: média/baixa.

Smells acumulados **viram** bugs ao longo do tempo — é dívida técnica vencendo juros.

As quatro famílias de Fowler

Bloaters

Long Method

Large Class

Long Parameter List

OO Abusers

Switch Statements

Temporary Field

Refused Bequest

Change Preventers

Divergent Change

Shotgun Surgery

Feature Envy

Dispensables

Duplicated Code

Dead Code

Speculative Generality

Veremos um representante de cada família + os clássicos.

- **Bloaters** — coisas que *incham* (métodos, classes, listas).
- **OO Abusers** — recursos de OO usados de forma errada (ou não usados).
- **Change Preventers** — estruturas que *dificultam* mudanças futuras.
- **Dispensables** — coisas que *não precisariam estar lá* (duplicação, código morto, abstração à toa).

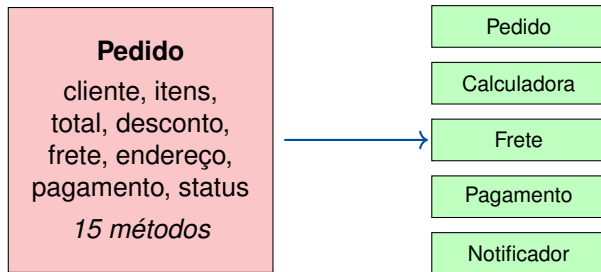
Bloaters

Sintomas

Mais de ~20 linhas, muitos níveis de indentação, comentários explicando *blocos* de código.

```
1 def processar_pedido(p):
2     if not p.cliente: raise ValueError("sem cliente")
3     if not p.itens:   raise ValueError("sem itens")
4     total = 0
5     for i in p.itens: total += i.preco * i.qtd
6     if total > 1000: total *= 0.9
7     msg = f"Pedido {p.id}: R$ {total}"
8     smtp.send(p.cliente.email, msg)
9     return total
```

Refatoração de combate: *Extract Function* — veremos a versão depois na Parte II.



Refatoração: *Extract Class* — separar por responsabilidade (SRP).

```
1 def criar_pedido(nome, cpf, email, rua, numero,  
2                 cidade, cep, produto, qtd, preco,  
3                 cupom, forma_pagto):  
4     ...
```

Sintoma: mais de 3–4 parâmetros, especialmente quando vários aparecem juntos (*data clumps*).

Refatoração: *Introduce Parameter Object* — agrupar em `Cliente`, `Endereco`, `ItemPedido`, `Pagamento`.

**Dispensables, Change
Preventers e OO Abusers**

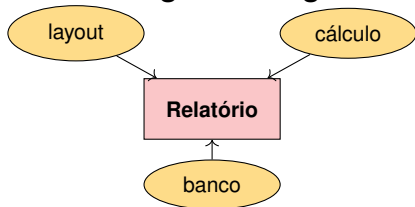
O problema

A mesma estrutura aparece em mais de um lugar. Mudança em um exige caçar e mudar os outros.

```
1 def preco_a(x): return x if x > 0 else -x
2 def preco_b(y): return y if y > 0 else -y
3 def preco_c(z): return z if z > 0 else -z
4 # depois: def preco(v): return abs(v)
```

Refatorações de combate: *Extract Function*, *Pull Up Method* (para superclasse), *Extract Class* (utilitária).

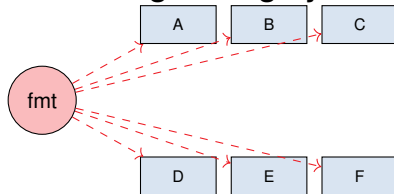
Divergent Change



Mesma classe muda por razões diferentes (viola SRP).

Refatoração: *Extract Class.*

Shotgun Surgery



Uma mudança obriga mexer em muitas classes.

Refatoração: *Move Method/Field.*


```
1 def calcular_frete(pedido):
2     if pedido.tipo == "NORMAL": return pedido.peso * 5.0
3     elif pedido.tipo == "EXPRESSO": return pedido.peso * 10.0
4     elif pedido.tipo == "SEDEX": return pedido.peso * 15.0
5     else: raise ValueError(pedido.tipo)
```

Por que cheira mal? adicionar um tipo novo exige caçar *todos* os switches — é *Shotgun Surgery* esperando para acontecer.

Refatoração: *Replace Conditional with Polymorphism* (uma classe por tipo de envio).

Smell	Refatoração de combate
Long Method	Extract Function
Large Class	Extract Class
Long Parameter List	Introduce Parameter Object
Duplicated Code	Extract Function / Pull Up
Divergent Change	Extract Class
Shotgun Surgery	Move Method / Field
Switch Statements	Replace Conditional w/ Polymorphism

O que é refração

Definição (Fowler, 2018)

Refatorar é reestruturar código existente alterando sua estrutura interna sem mudar seu comportamento externo observável.

Intuição

Como reorganizar uma cozinha enquanto se cozinha: você muda onde as coisas ficam, mas o jantar continua sendo o mesmo — e a cada passo você verifica que ainda está cozinhando.

Catálogo de refatorações básicas

Quatro refatorações que cobrem 80% dos casos

Extract Function

trecho coeso → função com nome próprio

Rename

trocar nome obscuro por nome que revela intenção

Inline

indireção desnecessária → código no lugar

Introduce Parameter Object

grupo de parâmetros recorrentes → objeto



Catálogo completo: refactoring.guru

```
1 def imprimir_extrato(cliente, lancamentos):
2     print(f"Cliente: {cliente.nome}")
3     print("-" * 30)
4     total = 0
5     for l in lancamentos:
6         total += l.valor
7     print(f"Total: R$ {total:.2f}")
```

O cálculo do total está *escondido* dentro de uma função de impressão. O nome da função mente sobre o que ela faz.

```
1 def calcular_total(lancamentos):
2     return sum(l.valor for l in lancamentos)
3
4 def imprimir_extrato(cliente, lancamentos):
5     print(f"Cliente: {cliente.nome}")
6     print("-" * 30)
7     total = calcular_total(lancamentos)
8     print(f"Total: R$ {total:.2f}")
```

Cada função tem *um* motivo para mudar. `calcular_total` agora é testável isoladamente e reutilizável.

1. Identifique um trecho coeso com responsabilidade própria.
2. Crie uma função nova com nome que *revele a intenção*.
3. Copie o trecho para a nova função; declare parâmetros para cada variável usada de fora.
4. Substitua o trecho original por uma chamada à nova função.
5. **Rode os testes.** Se passarem, *commit*.

Passos pequenos + testes a cada passo = confiança.

Quando refatorar

Regra dos três: duplicou pela terceira vez? Extraia.

Antes de adicionar feature: prepare o terreno onde a nova funcionalidade vai entrar (*“make the change easy, then make the easy change”* – Kent Beck).

Ao corrigir um bug: se entender o bug foi difícil, refatore para que o próximo na mesma região seja óbvio.

- Código que está prestes a ser *descartado*.
- Sem *rede de testes* cobrindo o trecho — escreva testes primeiro.
- Misturado com mudança de comportamento no mesmo *commit* (separe: ou refatora, ou muda comportamento, nunca os dois juntos).
- Sob pressão extrema de prazo, sem combinar com o time.

Estudo de caso

```
1 def calc(itens, t):
2     s = 0
3     for i in itens:
4         s = s + i[0] * i[1]
5     if t == "vip":
6         s = s * 0.9
7     elif t == "func":
8         s = s * 0.8
9     return s
```

Cheiros: nomes opacos (s, i, t), tupla anônima como “item”, desconto misturado com soma, if/elif sobre *strings mágicas*.

```
1 DESCONTO = {"vip": 0.10, "func": 0.20}
2
3 def subtotal(itens):
4     return sum(item.preco * item.qtd for item in itens)
5
6 def calcular_total(itens, perfil="comum"):
7     return subtotal(itens) * (1 - DESCONTO.get(perfil, 0))
```

Aplicamos: *Rename*, *Extract Function* (`subtotal`), *Replace Conditional with Lookup* (dicionário `DESCONTO`) e *Introduce Parameter Object* implícito (`item.preco` em vez de `i[0]`).

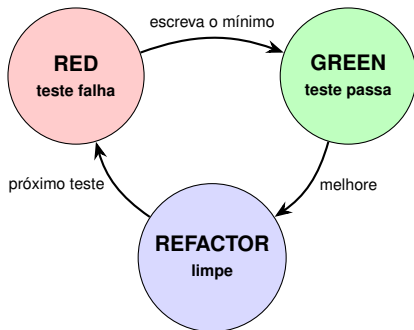
Ciclo Red–Green–Refactor

Problema:

Refatorar “no instinto” quebra código.
Mudanças grandes misturam alteração de *estrutura* com alteração de *comportamento* — e bugs se escondem.

Solução:

Um **ciclo curto e disciplinado** em que cada passo é pequeno, verificável por testes e reversível.
Red–Green–Refactor.



- **Red:** teste que falha define o que o código deve fazer.
- **Green:** faça passar do jeito mais simples.
- **Refactor:** com a rede verde, melhore o design.

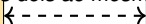
Chapéu 1: Adicionar Função

Mudar *comportamento*.
Não mexer em estrutura.
Testes novos podem falhar.

Chapéu 2: Refatorar

Mudar *estrutura*.
Não mexer em comportamento.
Testes **continuam** passando.

nunca os dois ao mesmo tempo



Red — teste primeiro:

```
1 def test_total_com_desconto():  
2     assert total([10, 20, 30], desconto=0.1) == 54.0
```

Green — o mínimo que faz passar:

```
1 def total(itens, desconto):  
2     return sum(itens) * (1 - desconto)
```

Refactor — nomes e responsabilidades:

```
1 def _aplicar_desconto(valor, taxa):  
2     return valor * (1 - taxa)  
3  
4 def total(itens, desconto=0.0):  
5     return _aplicar_desconto(sum(itens), desconto)
```

Testes com `pytest`

Definição

pytest é um framework de testes que descobre automaticamente funções `test_*` em arquivos `test_*.py`, executa-as, e relata falhas com introspecção detalhada da expressão `assert`.

Intuição

Você escreve Python comum (`def + assert`); o framework cuida da descoberta, do relatório e da rede de segurança que sustenta o ciclo Red–Green–Refactor.

```
1  # arquivo: test_carrinho.py
2  from carrinho import total
3
4  def test_total_lista_vazia():
5      assert total([]) == 0
6
7  def test_total_soma_itens():
8      assert total([1, 2, 3]) == 6
```

Execução:

```
1  $ pytest -v
2  test_carrinho.py::test_total_lista_vazia PASSED
3  test_carrinho.py::test_total_soma_itens PASSED
```

- **Fixtures** (`@pytest.fixture`) — preparam dados/estado reutilizáveis.
- **Parametrização** (`@pytest.mark.parametrize`) — um teste, vários casos.
- **Exceções esperadas** (`with pytest.raises(...)`).
- **Descoberta automática** — arquivos `test_*.py`, funções `test_*`.


```
1 import pytest
2
3 @pytest.mark.parametrize("entrada, esperado", [
4     ([], 0),
5     ([1, 2], 3),
6     ([10]*5, 50),
7 ])
8 def test_total(entrada, esperado):
9     assert total(entrada) == esperado
10
11 def test_sacar_mais_que_saldo_falha():
12     c = Conta(saldo=100)
13     with pytest.raises(SaldoInsuficienteError):
14         c.sacar(150)
```

Um teste parametrizado roda várias vezes (uma falha não impede as outras);
`pytest.raises` *documenta* o contrato de exceções.

Estilo: PEP 8

Definição

PEP 8 (*Python Enhancement Proposal 8*) é o documento que padroniza a aparência do código Python: indentação, nomenclatura, comprimento de linha, espaçamento em torno de operadores, organização de `imports`, entre outros.

Intuição — “a hobgoblin of little minds”

Consistência importa, mas não é cega: se a PEP 8 atrapalha a legibilidade num caso específico, ignore-a *nesse* caso. Consistência local > consistência global > PEP 8 literal.

Indentação: 4 espaços por nível (nunca tab).

Linha: máx. 79 caracteres de código, 72 de comentário.

Nomes: snake_case (funções), PascalCase (classes), MAIUSCULAS (constantes)

Imports: um por linha, no topo, agrupados (stdlib, terceiros, locais)

Espaçamento: 1 espaço em torno de =, exceto em argumentos default

Comparações: is None (não == None); if x: para coleções vazias.

Antes — viola convenções:

```
1 import os,sys
2 def CalcularMedia( numeros ):
3     Total=0
4     for N in numeros:Total=Total+N
5     return Total/len( numeros )
```

Depois — de acordo com a PEP 8:

```
1 import os
2 import sys
3
4
5 def calcular_media(numeros):
6     return sum(numeros) / len(numeros)
```

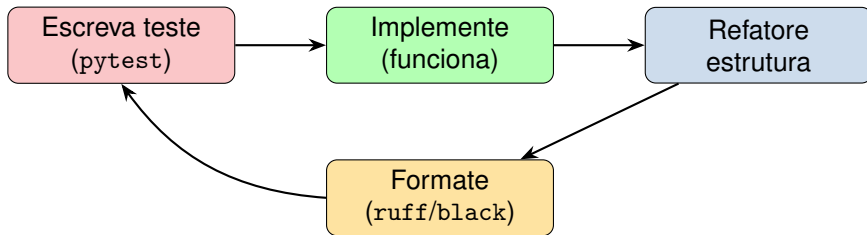
Mesmo comportamento, leitura imediata. É uma refatoração: estrutura mudou, comportamento não.

- `ruff` — linter rapidíssimo, agrupa `flake8` e mais.
- `black` — formatador opinativo, padroniza tudo.
- `isort` — ordena imports.

```
1 $ ruff check src/  
2 $ black src/  
3 $ pytest
```

Pipeline típico: formatar → lintar → testar.

Integrando os três



- Cheiros (Parte I) disparam refatorações nomeadas (Parte II).
- Testes garantem que a refatoração não muda comportamento.
- PEP 8 garante que o código refatorado é legível pela equipe.

Pratique

- Identifique 3 cheiros em um projeto seu.
- Cubra-os com testes `pytest` antes de mexer.
- Aplique a refatoração de combate em passos pequenos.
- Rode `ruff` + `black` ao final.

Leituras recomendadas

- Fowler (2018), *Refactoring*, caps. 1 e 3.
- Beck (2003), *TDD by Example*, parte I.
- Martin (2008), *Clean Code*.
- PEP 8 — peps.python.org/pep-0008.

“Make it work, make it right, make it fast.” — Kent Beck



Kent Beck.

Test-Driven Development: By Example.

Addison-Wesley, 2003.



Martin Fowler.

Refactoring: Improving the Design of Existing Code.

Addison-Wesley, 2 edition, 2018.



pytest Development Team.

pytest documentation, 2024.

Disponível em <https://docs.pytest.org/>.



Alexander Shvets.

Refactoring.guru — code smells catalog.

<https://refactoring.guru/refactoring/smells>, 2024.